

TD n°3 : Régression linéaire multivariée

Lors de mon précédent article, on a montré comment implémenter la régression linéaire *simple*, c’est-à-dire en utilisant une seule variable prédictive. Toutefois, une variable cible est généralement prédite non seulement avec une seule variable prédictive mais avec *plusieurs*. Dans ce cas, on parle de *Multivariate Regression* (*régression linéaire multivariée*). Dans cet article, on en implémentera une en utilisant les libraires de Machine learning de Python.

Exemple. Prenons, par exemple, la prédiction du prix d’une voiture. Le prix est la variable cible, les variables prédictives peuvent être : nombre de kilomètres au compteur, le nombre de cylindres, nombre de portes, *etc.* Toutes ces variables prédictives seront utilisées dans notre modèle de régression linéaire multivariée pour trouver une fonction prédictive.

Dans le cas de régression linéaire multivariée, la fonction prédictive s’écrira sous la forme :

$$y = a_0 + a_1 \cdot x + a_2 \cdot x^2 + \dots + a_n \cdot x^n$$

où :

- $a_0, a_1, \dots, a_n$ sont les *coefficients* (constantes) de la fonction prédictive ;
- x_i avec $1 \leq i \leq n$ représente la $i^{\text{ème}}$ variable prédictive ;
- $X = (x_1, x_2, \dots, x_n)$ représente un vecteur/tableau de variables prédictives ; pour l’exemple de prédiction du prix de voiture, la taille du vecteur X sera égale à 3 (nombre de Km, nombre de cylindres, et le nombre de portes).

Maintenant qu’on sait à quoi ressemblera notre fonction prédictive ainsi que nos données, essayons d’appliquer ce concept dans un cas concret en le codant en Python.

Exercice 0 : Présentation du problème. Supposons qu’on souhaite vendre une maison qui se trouve à Portland aux États-Unis. Pour déterminer un bon prix, on dispose des données de prix de vente effectuées dans le passé dans la même ville. Ainsi, pour chaque maison vendue (une observation), on dispose des données suivantes :

1. La taille de la superficie en pieds² (Un pieds² fait environ 0,092 m²) ;
2. Le nombre de chambre de la maison ;
3. Le prix auquel elle a été vendue.

Le jeu de données comporte 47 enregistrements. Le fichier de données est sous forme Excel, et est téléchargeable depuis https://github.com/ybenzaki/multivariate_linear_regression_python/raw/master/Multivariate_Linear_Regression_dataset.xlsx

Exercice 1 : Chargement des données. Pour commencer, il faudra lire et charger les données contenues dans le fichier Excel. Python propose via sa librairie Pandas des classes et fonctions pour lire divers formats de fichiers notamment les fichiers Excel.

1. Importez la librairie Pandas des classes et fonctions pour lire divers formats de fichiers, notamment les fichiers Excel.
2. Utilisez la fonction `read_excel()` pour renvoyer un DataFrame, c’est-à-dire un tableau de 3 dimensions contenant respectivement : la taille de la maison, le nombre de chambres et le prix auquel elle a été vendue.

3. Utilisez la fonction `head()` (avec les bons arguments !) soit pour afficher les premières lignes du Data-Frame, soit pour afficher toutes les lignes du DataFrame.
4. Récupérez l'ensemble de la variable cible y (`prix`) et des variables prédictives x_i (`taille_en_pieds_carre` et `nb_chambres`).

Exercice 2 : Représentation des données. Pour mieux comprendre les données, il est souvent utile de les visualiser. Ici, on a un jeu de données de 3 dimensions. Les données seront éparpillées dans un espace 3D (chaque ligne du dataset sera un triple (x_1, x_2, y)). On peut représenter les données dans un espace 3D avec la librairie `matplotlib` (cf. le TD 2).

Exercice 3 : Normalisation des données et feature Scaling. Notre exemple comporte des variables prédictives avec des ordres de grandeurs très différents. En effet, le nombre de chambre d'une maison est généralement compris entre 1 et 10 alors que la superficie se compte en quelques milliers de pieds².

Pour appliquer l'algorithme Multivariate Regression, il est nécessaire que les variables prédictives faisant partie du modèle prédictif soient du même ordre de grandeur. Généralement, il faut que la valeur de chaque variable prédictive soient compris (approximativement) entre -1 et 1 . Si certaines valeurs dépassent un peu (par exemple -2 , 1.5 ...) ce n'est pas très grave.

Pour ramener nos variables prédictives au même ordre de grandeur, nous appliquerons un procédé qui s'appelle : *features scaling*.

Note : On a pas bien détaillé la raison d'application du feature scaling. Pour le moment, reprenez juste que quand vous avez des variables prédictives avec des ordres de grandeurs différents, il faudra appliquer le feature scaling.

La librairie `Scikit learn` de Python propose plusieurs classes et méthodes pour faire de la préparation de données (*Data pre-processing*) pour les algorithmes de Machine Learning. Le package `sklearn.preprocessing` propose la classe `StandardScaler` qui permettra de faire du features scaling sur toutes nos variables prédictives. Pour cela, utilisez la méthode `fit_transform()`.

Exercice 4 : Apprentissage de l'algorithme par les données. On y est ! Tous les ingrédients sont là pour pouvoir appliquer l'algorithme de régression linéaire multivariée. Parmi les procédés mathématiques pour faire de la régression linéaire, il y a le *Ordinary Least Squares* (OLS).

OLS est une méthode pour estimer une variable cible dans un modèle de régression linéaire. Pour y parvenir, OLS va minimiser la somme des carrés des différences entre les réponses observées (de notre jeu de données) et ceux prédits par la fonction linéaire appliquée à nos variables prédictives de notre ensemble de données (cf. le TD 2).

Il vous suffira de savoir que c'est un estimateur que nous pourrions utiliser ici pour notre problème. Et tout ça pour dire que Python, propose une implémentation de OLS via son package `statsmodels.api`.

1. Utilisez la fonction `OLS()` du module `statsmodels.api` module pour réaliser une régression OLS. Il renvoie un objet OLS.
2. Utilisez la méthode `fit()` sur l'objet OLS pour faire "fitter" la ligne droite de régression linéaire aux données.
3. Utilisez la méthode `summary()` pour obtenir un tableau qui donne une description des résultats de régression. La colonne `coef` donne les valeurs des coefficients a_i de la fonction prédictive.
4. Répétez les points 1-3 après avoir utilisé la méthode `add_constant()` pour estimer la valeur de $a_0 \neq 0$.

Exercice 5 : Visualisation de la fonction de prédiction. Comme maintenant vous disposez d'un modèle prédictif, vous pouvez réutiliser le jeu de données pour "dessiner" la fonction prédictive qu'on a obtenu. Vu qu'on dispose d'un ensemble de données de 3 dimensions, la fonction obtenue sera également définie dans un espace 3D. Pour rappel, notre fonction prédictive est définie comme suit :

$$\text{prix_estime} = a_0 + a_1 \cdot \text{surface} + a_2 \cdot \text{nb_chambres}$$

Vous pouvez appliquer cette fonction sur les données qu'on a utilisées pour l'entraînement d'OLS, pour obtenir un ensemble de prédictions de prix :

```
def predict_price_of_house(taille_maison, nb_chambre):  
    #return 140.8611 * taille_maison + 1.698e+04 * nb_chambre
```

Et on dessine le tout dans un plan 3D, par ce code Python :

```
def predict_all(lst_sizes, lst_nb_chmbres):  
    predicted_prices = []  
    for n in range(0, len(Y)):  
        predicted_prices.append(predict_price_of_house(lst_sizes[n],  
            lst_nb_chmbres[n]))  
    return predicted_prices  
  
ax = fig.add_subplot(1, 2, 2, projection='3d')  
  
ax.plot_trisurf(df[\"taille-en-pieds-carre\"],  
    df[\"nb_chambres\"], predict_all(df[\"taille-en-pieds-carre\"],  
    df[\"nb_chambres\"]))  
  
plt.show()
```

Exercice 6 : Test de prédiction Vous pouvez utiliser notre modèle prédictif pour prédire le prix d'une maison qui n'est pas présente dans notre jeu de données. Estimons, le prix d'une maison avec 4500 pieds² et avec 5 chambres :

```
print predict_price_of_house(4500,5)  
# ce qui donne : 718774.95 $
```

On remarque que le prix prédit n'est pas très éloigné du prix de maisons à caractéristiques similaires. On peut bien dire qu'on a un bon estimateur de prix !